

ASSIGNMENT-16.3

Name: Madhumitha

HT. No: 2303A51625

Batch: 8

Task 1: Hospital Management Database

Scenario: Design a relational database for a Hospital Management System using Python and SQLite. The schema includes Doctors, Patients, and Appointments tables with proper constraints and foreign keys.

Tables: Doctors, Patients, Appointments

Queries Generated:

- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor.

Code & Output:

day5.py X

Users > mohammadmuneerahmed > Documents > training2.py > day5.py > ...

```
1 import sqlite3
2
3 conn = sqlite3.connect(":memory:")
4 cursor = conn.cursor()
5
6 cursor.executescript("""
7 CREATE TABLE Doctors (
8     doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
9     name TEXT NOT NULL,
10    specialization TEXT,
11    phone TEXT UNIQUE
12 );
13
14 CREATE TABLE Patients (
15     patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
16     name TEXT NOT NULL,
17     dob DATE,
18     gender TEXT CHECK(gender IN ('M', 'F', 'Other')),
19     phone TEXT UNIQUE
20 );
21
22 CREATE TABLE Appointments (
23     appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
24     doctor_id INTEGER NOT NULL,
25     patient_id INTEGER NOT NULL,
26     appointment_date DATE NOT NULL,
27     notes TEXT,
28     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
29     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
30 );
31
32 INSERT INTO Doctors (name, specialization, phone) VALUES
33     ('Dr. Smith', 'Cardiology', '9001'),
34     ('Dr. Jones', 'Neurology', '9002');
35
36 INSERT INTO Patients (name, dob, gender, phone) VALUES
37     ('Alice', '1990-05-10', 'F', '8001'),
38     ('Bob', '1985-03-22', 'M', '8002'),
39     ('Carol', '2000-11-01', 'F', '8003');
40
41 INSERT INTO Appointments (doctor_id, patient_id, appointment_date, notes) VALUES
42     (1, 1, '2025-01-10', 'Routine checkup'),
43     (1, 2, '2025-01-12', 'Follow-up'),
44     (2, 3, '2025-01-15', 'Initial consultation'),
45     (1, 3, '2025-01-20', 'ECG Test');
46 """)
47
48 print("Appointments for Dr. Smith:")
49 cursor.execute("""
50     SELECT a.appointment_id, p.name, a.appointment_date, a.notes
51     FROM Appointments a
52     JOIN Patients p ON a.patient_id = p.patient_id
53     WHERE a.doctor_id = 1
54 """)
55 for row in cursor.fetchall():
56     print(row)
57
58 print("\nHistory for Patient ID 3:")
59 cursor.execute("""
60     SELECT p.name, d.name, a.appointment_date, a.notes
61     FROM Appointments a
62     JOIN Patients p ON a.patient_id = p.patient_id
63     JOIN Doctors d ON a.doctor_id = d.doctor_id
64     WHERE a.patient_id = 3
65 """)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"

```
● mohammadmuneerahmed@Muneers-MacBook-Air ~ % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Appointments for Dr. Smith:
(1, 'Alice', '2025-01-10', 'Routine checkup')
(2, 'Bob', '2025-01-12', 'Follow-up')
(4, 'Carol', '2025-01-20', 'ECG Test')
```

```
History for Patient ID 3:
('Carol', 'Dr. Jones', '2025-01-15', 'Initial consultation')
('Carol', 'Dr. Smith', '2025-01-20', 'ECG Test')
```

```
Total Patients per Doctor:
('Dr. Smith', 3)
('Dr. Jones', 1)
```

● mohammadmuneerahmed@Muneers-MacBook-Air ~ %

Observation: The schema was successfully normalized with foreign key constraints. SQL JOIN queries efficiently retrieved appointment lists and patient histories. GROUP BY with COUNT gave per-doctor patient totals.

Task 2: Online Shopping Database

Scenario: Design a database for an E-commerce platform with Users, Products, Orders, and OrderDetails tables. Generate analytical queries for business insights.

Tables: Users, Products, Orders, OrderDetails

Queries Generated:

- Retrieve all orders by a user.
- Find the most purchased product.
- Calculate total revenue in a given month.

Code & Output:

day5.py

Users > mohammadmuneerahmed > Documents > training2.py > day5.py > ...

```
1 import sqlite3
2
3 conn = sqlite3.connect(':memory:')
4 cursor = conn.cursor()
5
6 cursor.executescript("""
7 CREATE TABLE Users (
8     user_id INTEGER PRIMARY KEY AUTOINCREMENT,
9     name TEXT NOT NULL,
10    email TEXT UNIQUE NOT NULL
11 );
12
13 CREATE TABLE Products (
14     product_id INTEGER PRIMARY KEY AUTOINCREMENT,
15     name TEXT NOT NULL,
16     price REAL NOT NULL CHECK(price >= 0),
17     stock INTEGER DEFAULT 0
18 );
19
20 CREATE TABLE Orders (
21     order_id INTEGER PRIMARY KEY AUTOINCREMENT,
22     user_id INTEGER NOT NULL,
23     order_date DATE NOT NULL,
24     FOREIGN KEY (user_id) REFERENCES Users(user_id)
25 );
26
27 CREATE TABLE OrderDetails (
28     detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
29     order_id INTEGER NOT NULL,
30     product_id INTEGER NOT NULL,
31     quantity INTEGER NOT NULL CHECK(quantity > 0),
32     unit_price REAL NOT NULL,
33     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
34     FOREIGN KEY (product_id) REFERENCES Products(product_id)
35 );
36
37 INSERT INTO Users (name, email) VALUES
38     ('Alice', 'alice@mail.com'),
39     ('Bob', 'bob@mail.com');
40
41 INSERT INTO Products (name, price, stock) VALUES
42     ('Laptop', 75000, 10),
43     ('Phone', 25000, 50),
44     ('Headset', 3000, 100);
45
46 INSERT INTO Orders (user_id, order_date) VALUES
47     (1, '2025-03-05'),
48     (1, '2025-03-18'),
49     (2, '2025-03-20');
50
51 INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price) VALUES
52     (1, 1, 1, 75000),
53     (1, 3, 2, 3000),
54     (2, 2, 1, 25000),
55     (3, 2, 2, 25000),
56     (3, 3, 1, 3000);
57 """)
58
59 print("Orders by Alice:")
60 cursor.execute("""
61     SELECT o.order_id, o.order_date, p.name, od.quantity, od.unit_price
62     FROM Orders o
63     JOIN OrderDetails od ON o.order_id = od.order_id
64     JOIN Products p ON od.product_id = p.product_id
65     WHERE o.user_id = 1
66 """)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
● mohammadmuneerahmed@Muneers-MacBook-Air ~ % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Orders by Alice:
(1, '2025-03-05', 'Laptop', 1, 75000.0)
(1, '2025-03-05', 'Headset', 2, 3000.0)
(2, '2025-03-18', 'Phone', 1, 25000.0)

Most Purchased Product:
('Headset', 3)

Total Revenue in March 2025:
159000.0
❖ mohammadmuneerahmed@Muneers-MacBook-Air ~ %
```

Observation: The e-commerce schema demonstrated multi-table JOIN operations. Aggregate functions (SUM, COUNT) with GROUP BY enabled revenue calculation and product popularity analysis. Normalization prevented data redundancy.

Task 3: Library Management Database

Scenario: Design a schema for a Library Management System with Books, Members, and Loans tables. Implement indexing for faster query execution.

Tables: Books, Members, Loans

Indexing Strategy: Indexes created on book_id, member_id, and loan/return dates.

Queries Generated:

- Retrieve all books currently issued (return_date IS NULL).
- Find overdue books (loan date > 30 days ago).
- Count number of books loaned by each member.

Code & Output:

Users > mohammadmuneerahmed > Documents > training2.py > day5.py > ...

```
1 import sqlite3
2 from datetime import date, timedelta
3
4 conn = sqlite3.connect(":memory:")
5 cursor = conn.cursor()
6
7 cursor.executescript("""
8 CREATE TABLE Books (
9     book_id INTEGER PRIMARY KEY AUTOINCREMENT,
10     title TEXT NOT NULL,
11     author TEXT NOT NULL,
12     isbn TEXT UNIQUE,
13     copies INTEGER DEFAULT 1
14 );
15
16 CREATE TABLE Members (
17     member_id INTEGER PRIMARY KEY AUTOINCREMENT,
18     name TEXT NOT NULL,
19     email TEXT UNIQUE
20 );
21
22 CREATE TABLE Loans (
23     loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
24     book_id INTEGER NOT NULL,
25     member_id INTEGER NOT NULL,
26     loan_date DATE NOT NULL,
27     return_date DATE,
28     FOREIGN KEY (book_id) REFERENCES Books(book_id),
29     FOREIGN KEY (member_id) REFERENCES Members(member_id)
30 );
31
32 CREATE INDEX idx_loans_book ON Loans(book_id);
33 CREATE INDEX idx_loans_member ON Loans(member_id);
34 CREATE INDEX idx_loans_dates ON Loans(loan_date, return_date);
35 """)
36
37 today = date.today()
38 overdue = (today - timedelta(days=35)).isoformat()
39 recent = (today - timedelta(days=5)).isoformat()
40
41 cursor.executescript(f"""
42 INSERT INTO Books (title, author, isbn) VALUES
43     ('Python Basics', 'Guido', 'ISBN001'),
44     ('Data Structures', 'Cormen', 'ISBN002'),
45     ('Clean Code', 'Martin', 'ISBN003');
46
47 INSERT INTO Members (name, email) VALUES
48     ('Alice', 'alice@lib.com'),
49     ('Bob', 'bob@lib.com');
50
51 INSERT INTO Loans (book_id, member_id, loan_date, return_date) VALUES
52     (1, 1, '{recent}', NULL),
53     (2, 1, '{overdue}', NULL),
54     (3, 2, '{recent}', NULL),
55     (1, 2, '{overdue}', '{recent}');
56 """)
57
58 print("Currently Issued Books:")
59 cursor.execute("""
60     SELECT b.title, m.name, l.loan_date
61     FROM Loans l
62     JOIN Books b ON l.book_id = b.book_id
63     JOIN Members m ON l.member_id = m.member_id
64     WHERE l.return_date IS NULL
65 """)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
● mohammadmuneerahmed@Muneers-MacBook-Air ~ % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Orders by Alice:
(1, '2025-03-05', 'Laptop', 1, 7500.0)
(1, '2025-03-05', 'Headset', 2, 3000.0)
(2, '2025-03-18', 'Phone', 1, 2500.0)

Most Purchased Product:
('Headset', 3)

Total Revenue in March 2025:
159000.0
❖ mohammadmuneerahmed@Muneers-MacBook-Air ~ %
```

Observation: The library schema used julianday() for date arithmetic to detect overdue books. Composite indexes on date columns significantly improved query performance. NULL-check conditions correctly identified currently issued books.

Task 4: Query Optimization

Scenario: Compare an inefficient subquery-based SQL query with an optimized JOIN-based query using EXPLAIN QUERY PLAN to verify performance improvement.

Original Query (Subquery - Slow):

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE  
country='UK');
```

Optimized Query (JOIN - Fast):

```
SELECT b.* FROM Books b JOIN Authors a ON b.author_id = a.author_id WHERE a.country =  
'UK';
```

Code & Output:

EXPLAIN QUERY PLAN confirms the JOIN uses the index directly, making it significantly faster on large datasets.

Task 5: University Course Registration

Scenario: SR University needs a course registration system for students enrolling in different courses under faculty supervision.

Tables: Students, Courses, Faculty, Registrations

Relationships:

- Each student can register for multiple courses.
- Each course is taught by one faculty member.

Queries Generated:

- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.

Code & Output:

Users > mohammadmuneerahmed > Documents > training2.py > day5.py > ...

```
1 import sqlite3
2
3 conn = sqlite3.connect(":memory:")
4 cursor = conn.cursor()
5
6 cursor.executescript("""
7 CREATE TABLE Faculty (
8     faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
9     name TEXT NOT NULL,
10    department TEXT
11 );
12
13 CREATE TABLE Courses (
14     course_id INTEGER PRIMARY KEY AUTOINCREMENT,
15     title TEXT NOT NULL,
16     credits INTEGER DEFAULT 3,
17     faculty_id INTEGER,
18     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
19 );
20
21 CREATE TABLE Students (
22     student_id INTEGER PRIMARY KEY AUTOINCREMENT,
23     name TEXT NOT NULL,
24     roll_no TEXT UNIQUE NOT NULL,
25     email TEXT UNIQUE
26 );
27
28 CREATE TABLE Registrations (
29     reg_id INTEGER PRIMARY KEY AUTOINCREMENT,
30     student_id INTEGER NOT NULL,
31     course_id INTEGER NOT NULL,
32     reg_date DATE DEFAULT CURRENT_DATE,
33     UNIQUE (student_id, course_id),
34     FOREIGN KEY (student_id) REFERENCES Students(student_id),
35     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
36 );
37
38 INSERT INTO Faculty (name, department) VALUES
39     ('Prof. Rao', 'CSE'),
40     ('Prof. Meera', 'IT'),
41     ('Prof. Arjun', 'MATH');
42
43 INSERT INTO Courses (title, credits, faculty_id) VALUES
44     ('Data Structures', 4, 1),
45     ('DBMS', 3, 1),
46     ('Operating Systems', 3, 2),
47     ('Calculus', 4, 3),
48     ('Algorithms', 3, 1);
49
50 INSERT INTO Students (name, roll_no, email) VALUES
51     ('Alice', '23CS01', 'alice@sru.edu'),
52     ('Bob', '23CS02', 'bob@sru.edu'),
53     ('Carol', '23CS03', 'carol@sru.edu');
54
55 INSERT INTO Registrations (student_id, course_id) VALUES
56     (1, 1), (1, 2), (1, 3),
57     (2, 1), (2, 4),
58     (3, 1), (3, 2), (3, 5);
59 """)
60
61 print("Students in Data Structures:")
62 cursor.execute("""
63     SELECT s.name, s.roll_no, c.title
64     FROM Registrations r
65     JOIN Students s ON r.student_id = s.student_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"

● mohammadmuneerahmed@Muneers-MacBook-Air ~ % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"

```
Students in Data Structures:
('Alice', '23CS01', 'Data Structures')
('Bob', '23CS02', 'Data Structures')
('Carol', '23CS03', 'Data Structures')
```

```
Faculty Teaching More Than 2 Courses:
('Prof. Rao', 3)
```

```
Courses by Registration Count:
('Data Structures', 3)
('DBMS', 2)
('Algorithms', 1)
('Calculus', 1)
('Operating Systems', 1)
```

Observation: The university registration schema successfully enforced a UNIQUE constraint on (student_id, course_id) to prevent duplicate registrations. HAVING clause filtered faculty with more than 2 courses. ORDER BY with COUNT produced a ranked list of courses by popularity.